CPTS 360 Programming Assignment 2: CPU Scheduling Report

Isaiah Foster WSU ID: XXXXXXXX

1 Overall Design

The simulator (`scheduler.c`) reads n and the (A,B,C,M) quadruples from the file named on the command line. Each quadruple is parsed with `fscanf(" ( %u %u %u %u )")`, so any amount of whitespace is accepted inside and between the parentheses, and anything after the nth quadruple (the "comments") is never read. Every process gets a `processID` equal to its position in the input.

All three algorithms share one function, `simulate(algorithm, ...)`, which runs on a fresh copy of the input. It differs between algorithms in only two places: whether the running process can be preempted (RR only) and how the next process is taken from the ready queue (FIFO for FCFS and RR, minimum remaining time for SJF). Each process stores a state (*unstarted*, *ready*, *running*, *blocked*, *terminated*), the remaining CPU burst, the remaining I/O burst, the length of its last CPU burst, the RR quantum left, and counters for CPU time, I/O time and waiting time.

Time advances in integer cycles; cycle t covers $[t,t+1)$. Each iteration of the main loop does the following:

1. **Running process transitions.** If the process on the CPU has run for C cycles in total, it terminates with finishing time t and is appended to the finished list. Otherwise, if its CPU burst is exhausted, it blocks for burst $\times M$ cycles. Otherwise, under RR, if its quantum is exhausted it is preempted back to ready.
2. **Blocked and arriving processes.** Blocked processes whose I/O burst has reached zero, and unstarted processes with $A = t$, become ready.
3. **Ready queue insertion.** Every process that became ready in this cycle, whether it was preempted, finished I/O or just arrived, is gathered into a batch. The batch is sorted by the tie-breaking rule and then appended to the ready queue.
4. **Dispatch.** If the CPU is idle and the ready queue is non-empty, a process is selected. If it has no CPU burst left over from a preemption, a new burst is drawn with `randomOS(B)` and capped at the CPU time remaining (C minus cycles run). The RR quantum is reset to 2.
5. **Accounting.** The running process gains one CPU cycle, and its burst and quantum each drop by one. Each blocked process gains one I/O cycle, and each ready process gains one waiting cycle. If any process is blocked, the global count of I/O-busy cycles goes up by one. Then t is incremented.

The loop ends when all n processes have terminated. The last termination happens at the start of a cycle, and the counter is still incremented after it, so the overall finishing time is `CURRENT_CYCLE - 1`, as the provided `printSummaryData()` expects. CPU utilization is total CPU time divided by the finishing time. I/O utilization is the number of cycles in which at least one process was blocked, divided by the finishing time.

2 The Three Algorithms

FCFS. The ready queue is a FIFO array of process indices. Dispatch always takes the head, and the running process keeps the CPU until its burst ends (it blocks) or it terminates. A process that comes back from I/O goes to the tail of the queue, so the CPU is shared in order of when each process became ready.

RR (quantum 2). This uses the same FIFO queue and dispatch as FCFS. On dispatch the quantum is set to 2, and step 1 preempts the process when the quantum reaches zero and its burst is not yet finished. The preempted process keeps its remaining CPU burst, so no new random number is drawn when it next runs. Its eventual I/O burst is still computed from the full burst it originally drew (stored in `lastCPUBurst`).

SJF. This is not preemptive, but a process gives up the CPU when it blocks for I/O. Priority is the total CPU time remaining, $C-$ cycles run, not the length of the next burst. On dispatch, `dequeueReady()` scans the

whole ready queue for the process with the smallest remaining time and removes it. A running process is never interrupted by a shorter job that arrives later; the shorter job waits until the CPU becomes free.

3 Tie-Breaking

Simultaneous events for the running process. Step 1 uses an `if / else if / else if` chain in the order the spec requires: termination first, then blocking, then preemption. If a process's last CPU cycle also ends its burst and its quantum, it terminates and does not block. If its burst and quantum end together, it blocks and is not preempted.

Processes with equal priority. One comparison function, `favoredOver(a, b)`, implements the rule: the earlier arrival time A wins, and if the arrival times are equal, the lower `processID` (the process listed earlier in the input) wins. It is used in two places:

- **FCFS and RR:** all processes that become ready in the same cycle (preempted, returning from I/O or newly arrived) are collected into one batch. The batch is sorted with `favoredOver` before it is appended to the FIFO queue, so simultaneous insertions are ordered by $(A, \text{input order})$. Processes already in the queue keep their earlier positions.
- **SJF:** when two ready processes have the same remaining CPU time, `favoredOver` decides between them during the minimum scan. This makes the choice independent of the order in which processes entered the queue.

The rule breaks every tie, because no two processes share both the same A and the same input position. Only one process runs at a time, so at most one process can terminate in a given cycle, and the order of the finished list is therefore also fully determined.

4 Testing

The program reproduces `sample_io/output/summary/output-1` through `output-3` byte for byte. The samples only use $B = 1$, so I also checked RR preemption by hand on `2 (0 5 6 1) (0 5 6 1)`. That run draws bursts longer than the quantum, has a burst and a quantum end in the same cycle, and has a preempted process tie with a process returning from I/O; every metric matched the hand trace. Further inputs with unsorted arrival times, irregular spacing, trailing comments and $M = 0$ ran cleanly under AddressSanitizer and UndefinedBehaviorSanitizer.